Machine Learning for teens

By Hamid Sadeghi

Table of contents

- Overfitting happens **because of high variance**
- Variance formula in statistics
- Visualization of variance
- Overcome overfitting
  - Regularization
    - L1
    - L2
    - ElasticNet
  - Cross-Validation
    - K-fold vs Stratified K-Fold and cross_val_score
      - Balanced vs Imbalanced dataset
    - **Cross-validation doesn't prevent overfitting directly**
- score() vs accuracy_score()

Overfitting happens **because of high variance**

- High variance means the model is too sensitive to the training data. That cause overfitting or
  - High variance means the model performs well on training data but behaves unpredictably on test data
- Definition of Variance
  - In statistics
    - **Variance** is a statistical measure that describes how much the values in a dataset differ from the mean. It quantifies the **spread** or **dispersion** of the data.
  - In machine learning
    - high variance in a model means it's **too sensitive to training data**, leading to **overfitting**

Variance formula in statistics

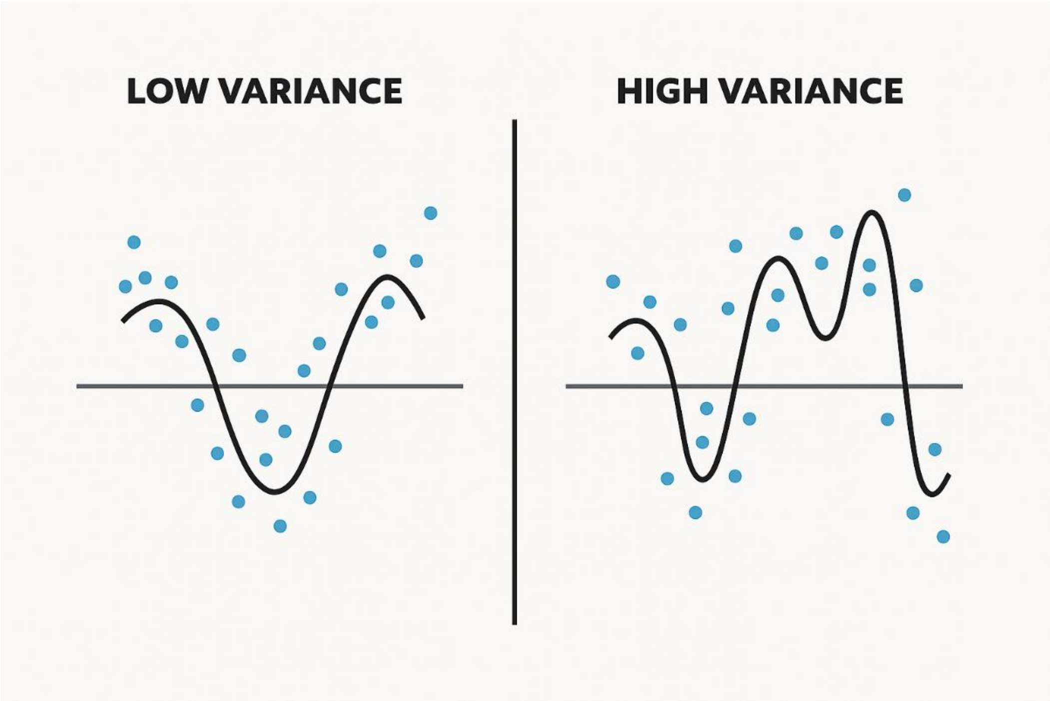
For a dataset with values $x_1, x_2, \ldots, x_n$ and mean μ:

$$\text{Variance} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

- x_i: each data point
- μ: mean of the dataset
- $(x_i - \mu)^2$: squared difference from the mean
- $\frac{1}{n}$: average over all data points

Visualization of variance

- When you have high variance, the data is very spread out from the mean. As a result, when your model tries to learn from the training data, it also captures noise — which leads to poor performance on other datasets



Ways to Overcome Overfitting

- **Feature Selection**
  - Simplify the model by removing irrelevant or noisy features.
- Regularization
  - Regularization works for linear models.
- Cross-Validation
  - KFold, StratifiedKFold, cross_val_score
- **Model Constraints** (for Trees)
  - Like our code
- Dimensionality Reduction
  - Reduce feature space like PCA

Ways to Overcome Overfitting

- Dropout
  - For neural networks
- Data Augmentation (for images/text)
  - Increase training diversity.
    - Flip, rotate, crop images
    - Synonym replacement for text
- Ensemble Methods
  - Combine models to reduce variance.
    - Bagging (Random Forest), Boosting (XGBoost), Stacking

Regularization

- Regularization is a technique used in machine learning to **reduce overfitting**
- When we add L1, L2, or ElasticNet regularization, we're no longer just trying to **fit the data perfectly**
- Get the script from here
- **Lasso (L1)** → will shrink and eliminate irrelevant features
- **Ridge (L2)** → will shrink coefficients but keep all features
- **ElasticNet** → combines both

Lasso (L1)

- It adds a penalty to the loss function based on the **absolute values** of the model's coefficients

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

$$\text{Loss} = \text{MSE} + \alpha \sum_{j=1}^{p} |\beta_j|$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- MSE: Mean Squared Error
- α: regularization strength
- β_j: model coefficients

Ridge (L2)

- Adds a penalty to the loss function based on the **squared magnitude** of coefficients

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\text{Loss} = \text{MSE} + \alpha \sum_{j=1}^{p} \beta_j^2$$

- Unlike Lasso, Ridge **never zeroes out** coefficients
- It's great when features are **correlated** or when you want **stability** over sparsity

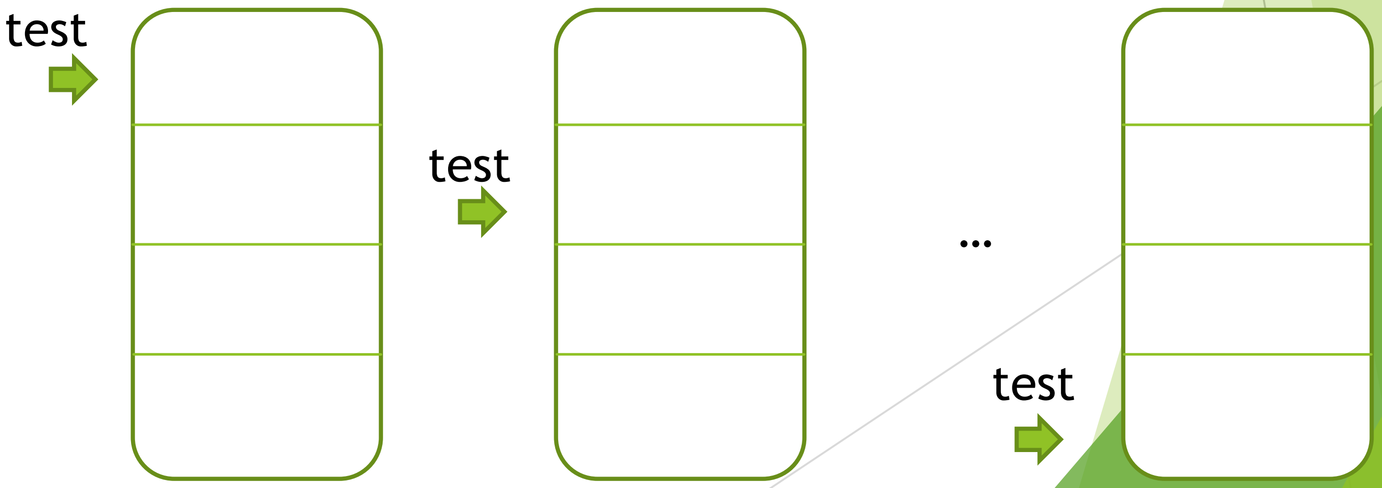
ElasticNet

- ElasticNet combines **L1 (Lasso)** and **L2 (Ridge)** penalties

$$\text{Loss} = \text{MSE} + \alpha\left(\lambda \sum |\beta_j| + (1-\lambda)\sum \beta_j^2\right)$$

Cross-Validation

- When we use the data splitting method, we usually get different splits each time — unless we set a fixed random seed.
- We divide our data into **K = 4 folds**, as shown in the diagram.
- The problem with the **train-test split** method is that it gives you **different splits each time**, unless you set a fixed random seed. As a result, your model may produce **different scores on each run**, making evaluation less reliable.
- Solution: Cross-Validation
  - K-fold
  - Stratified k-fold
  - cross_val_score
- Link to the code



K-fold vs Stratified K-Fold

- Use hand-written digit from sklearn . See the code
- If target is continuous (regression) use k-fold because it doesn’t care about y
- If target is discrete use stratified k-fold . Y is important . Useful for when you have especially imbalanced dataset
  - Balanced Dataset
    - you have the same number in each class
  - Imbalanced Dataset
    - You don't have the same number in each class

Dataset Type	Class 0 Count	Class 1 Count
Balanced	500	500
Imbalanced	900	100

- We use cross_val_score to
  - to avoid manually choosing between K-Fold and StratifiedKFoldNot
  - to get averaged scores automatically

When use k-fold and Stratified K-Fold

- **Continuous target** → use **K-Fold**
- **Discrete target** → use **StratifiedKFold** (especially if imbalanced)

Cross-validation doesn’t prevent overfitting directly

- Cross-validation doesn’t prevent overfitting directly — it helps you detect if it’s happening.

score() vs accuracy_score()

- When use each. See the code
- .score
  - can be used for **both classification and regression:**
    - Classification
      - returns **accuracy (same value as accuracy_score)**
    - Regression
      - returns **R^2 score**
- accuracy_score
  - Used for **classification only**

Assignment

- Use breast cancer dataset from sklearn do these(from sklearn.datasets import load_breast_cancer)
  - Use Logistic Regression
  - Overfitting
    - Check overfitting
  - Regularization
    - Compare Logistic Regression with and without regularization
      - L!
      - L2
      - ElasticNet
  - Cross-Validation
    - Use cross_val_score to get average score
  - Evaluation Metrics
    - Compare .score() vs accuracy_score() — do they match?